Workout Builder

Three-Sprint Roadmap: HTML to Native iOS

From single-file PWA to native iOS app with Apple Watch and HealthKit integration.

Sprint	Theme	Stack	Ship
1	Memory - History, RPE, UX polish	HTML + localStorage	GitHub Pages
2	Shell - Native iOS wrapper + iCloud sync	SwiftUI + WKWebView	TestFlight
3	Body - Apple Watch + HealthKit	WatchKit + HealthKit	App Store

Guiding principle: Each sprint ships a usable product. Sprint 1 improves the existing app. Sprint 2 wraps it in native chrome without rewriting the core. Sprint 3 extends into Apple's ecosystem. At no point does the app stop working or regress.

Architecture progression

Layer	Sprint 1	Sprint 2	Sprint 3
UI	HTML/CSS/JS (single file)	Same HTML in WKWebView	Same + WatchOS companion
State	localStorage	Swift bridge reads/writes iCloud	Same + HealthKit store
Persistence	Per-device JSON in browser	CloudKit via NSUbiquitousKVStore or file sync	Same + HKWorkoutSession
Distribution	GitHub Pages PWA	TestFlight (personal)	App Store or TestFlight

Sprint 1: Memory

Give the app a brain. Every session remembered, every pattern visible.

Stack	HTML + CSS + JS (single file, same repo). localStorage for persistence. Zero infrastructure changes. Ships to GitHub Pages as before.

Constraint	Everything in this sprint must work in the existing single HTML file. No server, no build step, no native code. If it can't ship with git push, it doesn't belong here.

1A. Session History + Persistence

The highest-impact change. The app currently forgets everything the moment you finish. This makes it remember.

Data model

```
CompletedSession { id: string (crypto.randomUUID), date: string (ISO 8601), type: 'hiit' | 'strength' | 'conditioning' | 'functional', intensity: 'light' | 'moderate' | 'high', workSec: number, restSec: number, plannedDurationMin: number, actualDurationSec: number, exercises: ExerciseRecord[], rpe: number | null, // 1-10, captured post-session note: string | null, // free text, max 200 chars skippedCount: number, extraRestSec: number // total +10s taps accumulated } ExerciseRecord { name: string, category: string, section: 'warmup' | 'main' | 'cooldown', workSec: number, restSec: number, completed: boolean, // false if skipped weightKg: number | null // optional, strength mode only }
```

Storage implementation

- All sessions stored as JSON array in `localStorage.setItem('wk_sessions', JSON.stringify(sessions))`
- Read on app load, write on session completion
- Include an export/import function (download/upload JSON file) as a manual backup and cross-device bridge until Sprint 2
- Cap storage at 500 sessions (~200KB) with oldest-first eviction if exceeded

History screen

New screen accessible from a 'History' button on the setup page. Two views:

- **Calendar heatmap:** 12-week grid (like GitHub contribution graph). Each day is a cell coloured by session intensity. Empty days are dark. Tap a day to see that session's detail.
- **Session list:** Scrollable list below the heatmap. Each row shows: date, workout type badge, duration, RPE (if logged), exercise count. Tap to expand and see full exercise list.
- **Stats strip:** At the top - total sessions, current streak, most common workout type, average RPE. Simple counters, not charts.

UX integration

- Add a bottom nav bar with three items: **Build** (current setup), **History**, **Settings** (future use, placeholder for now)
- The nav bar appears on Setup and History screens only - not during workouts
- Existing generate-preview-timer-done flow is unchanged

1B. Post-Session Capture

Transform the Done screen from a dead end into a data collection point.

RPE input

- Row of 10 circular buttons (1-10) using the existing pill UI pattern
- Colour gradient: 1-3 green, 4-6 yellow, 7-8 orange, 9-10 red
- Label: "How hard was that?" - tap one number, it highlights
- Optional - user can skip straight to 'Save & Finish'

Notes input

- Single-line text input below RPE, placeholder 'Any notes?'
- 200 character cap, no validation beyond length
- Useful for logging things like 'left shoulder tight' or 'felt strong today'

Done screen flow

Done! stats at top (unchanged) -> RPE selector -> Notes input -> 'Save & Finish' button (saves session to localStorage and returns to Setup) -> secondary 'Skip' link below for users who don't want to log anything.

1C. Preview Screen Enhancements

Exercise reordering

- Add up/down arrow buttons to each exercise row, positioned between the exercise info and the delete button
- Tap up-arrow: swap exercise with the one above. Down-arrow: swap with below
- Simpler and more reliable than drag-and-drop on mobile Safari
- Arrows are small SVG icons matching the existing icon size

Exercise swap

- Tapping an exercise name opens a swap picker modal
- Modal shows alternative exercises from the same category (e.g., other lower-squat exercises)
- Tap an alternative to replace the current exercise
- Reuses the existing info modal UI pattern - consistent, no new components

Strength mode: rep targets

- When workout type is 'strength', display a suggested rep count on each exercise in the preview
- Derived from exercise difficulty: diff 3 = 3-5 reps, diff 2 = 6-8 reps, diff 1 = 10-12 reps

- During timer, strength mode uses count-up timer with 'Done' button instead of countdown
- Optional weight input (numeric keypad) appears when user taps 'Done' on a set

Sprint 1 deliverables checklist

Deliverable	Detail
Session history stored in localStorage	Data model, read/write, 500-session cap
History screen with calendar heatmap	12-week grid, session list, stats strip
Post-session RPE + notes capture	Optional inputs on Done screen
Exercise reorder (up/down arrows)	Preview screen enhancement
Exercise swap picker	Same-category alternatives modal
Strength mode rep targets + count-up timer	Timer behaviour change for one mode
Export/import JSON	Manual backup, cross-device bridge
Bottom nav bar	Build / History / Settings (placeholder)

Sprint 2: Shell

Wrap the web app in native iOS chrome. Add iCloud sync. Ship via TestFlight.

Stack	Xcode project with SwiftUI + WKWebView. The existing HTML is bundled into the app and loaded locally. Swift provides native bridges for persistence and platform features. Apple Developer account required ($99/year).
Key principle	The HTML/JS remains the app's brain and UI. Swift is a thin shell that provides capabilities the web platform can't: iCloud sync, native file access, background audio, and later HealthKit. The web layer calls Swift via window.webkit.messageHandlers. Swift calls JS via webView.evaluateJavaScript.

2A. Xcode Project Setup

Minimal SwiftUI app that loads the workout HTML in a WKWebView.

Project structure

```
WorkoutBuilder/ WorkoutBuilderApp.swift // @main entry, window setup ContentView.swift //
SwiftUI view hosting WKWebView WebViewBridge.swift // WKScriptMessageHandler - JS-to-Swift
bridge CloudSyncManager.swift // iCloud key-value or document sync Resources/ index.html //
Existing workout app (bundled) WorkoutBuilder.entitlements // iCloud capability Info.plist
```

ContentView.swift

- Hosts a WKWebView that loads index.html from the app bundle
- Configures WKUserContentController with message handlers for the JS-Swift bridge
- Handles navigation delegates to prevent external link escapes
- Sets webView.scrollView.bounces = false for native feel
- Full-screen, no native chrome - the HTML provides all UI

Build targets

- iOS 17+ (covers all devices that would realistically run this)
- iPhone and iPad universal
- No Mac Catalyst needed (GitHub Pages version still works on Mac)

2B. JavaScript-Swift Bridge

The bridge replaces localStorage with iCloud-backed persistence. The HTML/JS doesn't need to know whether it's running in Safari or in the native shell - the bridge is transparent.

Bridge protocol

JS calls Swift:

```
// JS side - save sessions to native storage
window.webkit.messageHandlers.saveData.postMessage({ key: 'wk_sessions', value:
JSON.stringify(sessions) }); // JS side - request data load
window.webkit.messageHandlers.loadData.postMessage({ key: 'wk_sessions' });
```

Swift calls JS:

```
// Swift side - deliver loaded data back to JS webView.evaluateJavaScript(
"window.nativeBridge.onDataLoaded('wk_sessions', \(jsonString))" )
```

Graceful degradation

- The HTML checks for `window.webkit.messageHandlers` on load
- If present (native app): use the bridge for all persistence
- If absent (Safari/GitHub Pages): fall back to localStorage
- Same HTML works in both environments - no forking

This means Sprint 1's localStorage implementation isn't throwaway work. It becomes the fallback path.

2C. iCloud Persistence + Cross-Device Sync

Session data syncs across all of Joe's Apple devices automatically.

Two options, one recommendation

Approach	Pros	Cons	Verdict
NSUbiquitousKeyValueStore	Zero setup, automatic sync, key-value API matches localStorage pattern exactly	1MB total limit (~2,500 sessions - plenty), no conflict resolution, no querying	Recommended
CloudKit + CKRecord	Unlimited storage, per-record sync, queryable, conflict resolution	Significant complexity, requires schema design, overkill for single-user session log	Overkill

NSUbiquitousKeyValueStore implementation

- CloudSyncManager.swift wraps NSUbiquitousKeyValueStore
- On save: write to both local (UserDefaults) and iCloud (NSUBKVS) simultaneously
- On app launch: check iCloud for newer data, merge if needed (latest-wins by session ID)
- Register for NSUbiquitousKeyValueStoreDidChangeExternallyNotification to handle incoming syncs
- Push updated data to JS via the bridge when remote changes arrive

Conflict strategy

- Sessions are append-only (you don't edit past sessions), so conflicts are rare
- Merge strategy: union of all session IDs. If same ID exists in both local and remote, keep the one with more data (non-null RPE wins over null)
- Last-write-wins for settings/preferences

2D. Native Polish

- **App icon:** Simple dark icon matching the app's colour scheme - dark background, accent-blue dumbbell or timer glyph
- **Launch screen:** Solid dark background (#0a0a0f) matching the app's bg, no splash image needed
- **Status bar:** Light content on dark background (already configured via meta tag)
- **Audio session:** Configure AVAudioSession for playback category so beeps and TTS work when silent switch is on and when app is briefly backgrounded
- **Haptics:** Add subtle haptic feedback (UIImpactFeedbackGenerator) on phase transitions via the bridge - light tap on rest, medium on work start
- **Screen idle:** UIApplication.shared.isIdleTimerDisabled = true during workouts (replaces the JS wake lock which is unreliable on iOS)

Sprint 2 deliverables checklist

Deliverable	Detail
Xcode project with WKWebView	Loads existing HTML from bundle
JS-Swift bridge (saveData / loadData)	Transparent persistence layer
iCloud sync via NSUbiquitousKeyValueStore	Cross-device session history
Graceful degradation	HTML still works standalone in Safari
AVAudioSession configuration	Sound works with silent switch on
Native haptics on phase transitions	UIImpactFeedbackGenerator via bridge
Idle timer disabled during workouts	Native wake lock replacement
App icon + launch screen	Visual identity
TestFlight distribution	Personal testing across devices

Sprint 3: Body

Connect the app to Apple's health and fitness ecosystem. Watch companion. HealthKit integration. Activity rings.

Stack	Add WatchKit target to existing Xcode project. HealthKit framework on both iOS and watchOS. Watch Connectivity for phone-watch communication. This is the most technically complex sprint.

Key principle	The Watch app is a companion, not a replacement. It shows the current exercise and timer, receives haptic cues, and can pause/skip/extend rest. All workout generation and history stays on the phone. The Watch is a remote display with a heart rate sensor.

3A. HealthKit Integration (iPhone)

Every completed workout is written to Apple Health. Shows up in Fitness app. Closes Activity rings. Syncs across all Apple devices via iCloud Health.

Workout type mapping

App workout type	HKWorkoutActivityType	Notes
Strength	.traditionalStrengthTraining	Maps cleanly
HIIT	.highIntensityIntervalTraining	Maps cleanly
Conditioning	.crossTraining	Best general match
Functional	.functionalStrengthTraining	Maps cleanly

Data written per session

- **HKWorkout:** activity type, start date, end date, duration, energy burned (estimated from MET values per exercise type and duration)
- **HKWorkoutEvent:** Individual exercise segments as workout events (pause/resume markers for rest periods)
- **Energy estimate:** Conservative MET-based calculation. Strength = 3.5 MET, HIIT = 8.0 MET, Conditioning = 6.0 MET, Functional = 5.0 MET. Multiply by duration and user weight (requested once, stored in settings).

Implementation

```
// HealthKitManager.swift func saveWorkout(session: CompletedSession) async throws { let config = HKWorkoutConfiguration() config.activityType = mapWorkoutType(session.type) let workout = HKWorkout( activityType: config.activityType, start: session.startDate, end: session.endDate, duration: session.actualDurationSec, totalEnergyBurned: estimateCalories(session), totalDistance: nil, metadata: ["WorkoutBuilder": true] ) try await healthStore.save(workout) }
```

Permission flow

- Request HealthKit authorisation on first session completion, not on app launch

- Request write-only for: HKWorkoutType, HKQuantityType.activeEnergyBurned
- If denied: sessions still save to iCloud, just not to Health. Show a subtle banner, not an error.
- Add a toggle in Settings to disable HealthKit sync if the user doesn't want it

3B. Apple Watch Companion App

A glanceable remote display for the active workout. Not a standalone workout app.

Watch app scope

The Watch app does five things and nothing more:

- **Display:** Current exercise name, phase (WORK/REST), countdown timer, exercise number
- **Haptics:** Strong tap on work start, gentle tap on rest start, triple tap on 3-2-1 countdown
- **Controls:** Pause/resume button, skip button, +10s rest button
- **Heart rate:** Live heart rate display from HealthKit streaming during workout
- **Summary:** Post-workout summary with duration, exercise count, avg heart rate, calories

What the Watch does NOT do

- No workout generation - that happens on the phone
- No exercise library or info modals - screen is too small
- No history browsing - phone only
- No standalone mode (requires phone connection to start)

Watch UI (SwiftUI)

```
// Single-screen workout view VStack { Text("WORK") // Phase label, green/blue
.font(.caption).bold() Text("Kettlebell Swing") // Exercise name .font(.title3).bold()
Text("2:34") // Countdown, large .font(.system(size: 48, weight: .thin, design: .rounded))
.monospacedDigit() Text("142 BPM") // Heart rate .font(.caption).foregroundColor(.red) HStack
{ // Controls Button("||") { pause() } Button("+10s") { addRest() } Button(">>") { skip() } }
Text("5 of 14") // Progress .font(.caption2).opacity(0.6) }
```

Phone-Watch communication

- **Framework:** WatchConnectivity (WCSession)
- **Phone to Watch:** sendMessage for real-time state updates during workout (exercise name, phase, time remaining, exercise index)
- **Watch to Phone:** sendMessage for control actions (pause, skip, +10s) - phone updates state, sends new state back
- **Workout start:** Phone sends full workout metadata (exercise list, timings) via transferUserInfo when user taps 'Start Workout'
- **Connection lost:** Watch shows 'Reconnecting...' and freezes display. Does not attempt to run the workout independently.

HKWorkoutSession (Watch)

The Watch starts an HKWorkoutSession when the workout begins. This:

- Gives the app priority CPU and sensor access
- Enables live heart rate streaming via HKLiveWorkoutBuilder
- Shows the green workout indicator on the watch face
- Prevents the Watch from sleeping during the workout
- Automatically contributes to Activity rings

When the workout ends, the Watch calls endCollection() and finishWorkout(). The workout data (including heart rate samples and calories) is saved to HealthKit on the Watch and syncs to the phone automatically.

3C. Enhanced Post-Workout Summary

With HealthKit data flowing, the Done screen gets richer.

- **Heart rate summary:** Avg, max, and time in zones (if Watch was connected)
- **Calories:** Actual from Watch sensors, or estimated if no Watch
- **Apple Health badge:** Small green checkmark and 'Saved to Health' confirmation
- **Activity ring contribution:** Show how many Move/Exercise minutes this session added
- All of this is below the existing RPE/notes capture - doesn't change that flow

Sprint 3 deliverables checklist

Deliverable	Detail
HealthKit write on session completion	Workout type, duration, energy burned
HealthKit permission flow	Request on first save, graceful denial handling
Watch companion app	Display, haptics, controls, heart rate
WatchConnectivity integration	Real-time phone-watch state sync
HKWorkoutSession on Watch	Live HR, Activity ring contribution
Enhanced Done screen	HR summary, calories, Health confirmation
Settings screen	Weight input, HealthKit toggle, about info
User weight input	Required for calorie estimation, stored in iCloud

Risks, Dependencies & Decisions

Risk	Sprint	Severity	Mitigation
Apple Developer account not active	2	Blocker	Enrol before Sprint 2 starts. $99/year. Needed for provisioning profiles, TestFlight, iCloud entitlements, and HealthKit.
localStorage 5MB limit on iOS Safari	1	Low	Session data is small (~80 bytes/session). 500 sessions = ~40KB. Not a real risk, but cap it anyway.
NSUbiquitousKeyValueStore 1MB limit	2	Low	Each session is ~200 bytes as compressed JSON. 1MB = ~5,000 sessions = ~3 years of daily training. Sufficient.
WKWebView JS bridge latency	2	Low	Message passing is async but sub-millisecond on modern devices. Not perceptible for save operations.
HealthKit authorisation denied	3	Medium	App must work fully without HealthKit. Treat it as an enhancement, not a dependency. Clear messaging if denied.
Watch connectivity drops mid-workout	3	Medium	Watch freezes display and shows reconnecting state. Phone continues running the workout. Watch resumes when connection restores.
App Store review for a WKWebView app	3	Medium	Apple sometimes rejects apps that are 'just a web wrapper.' Mitigated by native features: HealthKit, Watch app, haptics, iCloud. These are genuinely native capabilities.

Open Decisions

Decision	Options	Recommendation	Decide by
App Store distribution vs TestFlight-only	TestFlight (90-day builds, up to 100 testers) vs App Store (public, review process)	TestFlight-only unless sharing with others. Avoids review friction.	Sprint 2 start
Calorie estimation method	Fixed MET per workout type vs per-exercise MET vs skip calories entirely	Fixed MET per type. Simple, good enough, avoids false precision.	Sprint 3 start
Watch standalone mode	Watch can run workouts independently vs phone-required	Phone-required. Keeps Watch app simple. Revisit if user demand emerges.	Sprint 3 start
GitHub Pages sunset	Keep GitHub Pages version alive vs deprecate after native ships	Keep it. Zero maintenance cost and works as a fallback on non-Apple devices.	Never

---

The app's identity doesn't change across these three sprints. It's still 'open it, tap twice, start moving.' Each sprint adds a layer of memory, durability, and ecosystem integration without compromising the 10-second speed-to-sweat that

makes the app worth using.